LAB 5
DEVELOP SOFTWARE FOR NIOS II - PROCESSOR WITH TIMER (PART 2)

I. Objective

- Students study Lab 5 after doing all steps in Lab 4. The objective of this Lab is to provide for students the knowledge of programming with Nios II processor using C language. After study Lab 5 students can understand how to program some basic projects with timer using C language.

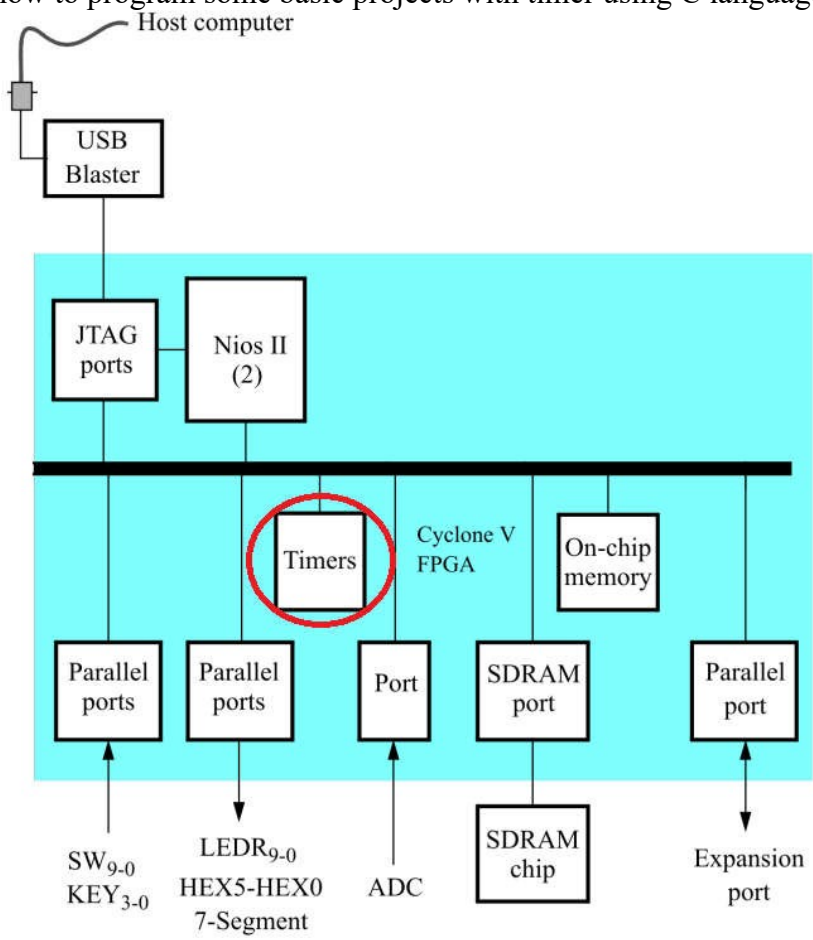


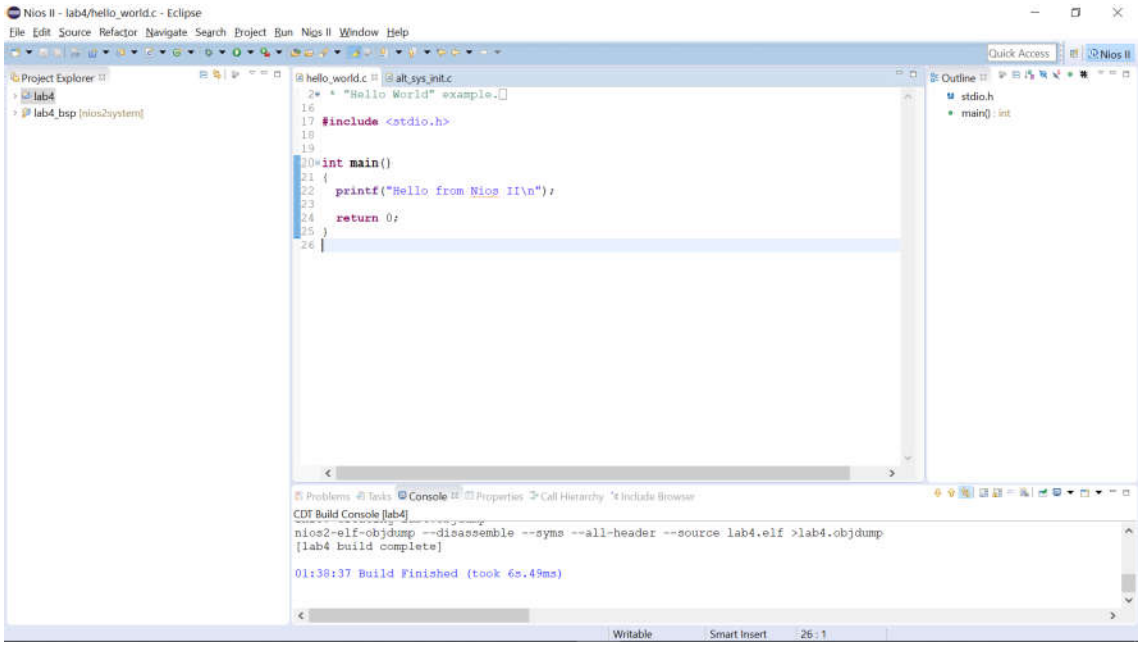
Figure 1: Connection Diagram of Timer Hardware in a project.

II. Preliminary information:

1. Summary of Lab 4

Lab 4 can be summarized into these steps:

- Use DE10-Standard Sytem Builder Tool to create a project with necessary peripherals.
- Use Qsys Tool to configure a system with Nios II processor and timer.
- Create HDL code with Qsys Tool and connect the system with peripherals using VHDL or Verilog on Quartus.
- Use Quartus 18.1 programmer to download .sof file to the development board (DE10- standard) via JTAG interface.
- Use Eclipse Software to program and run c code.



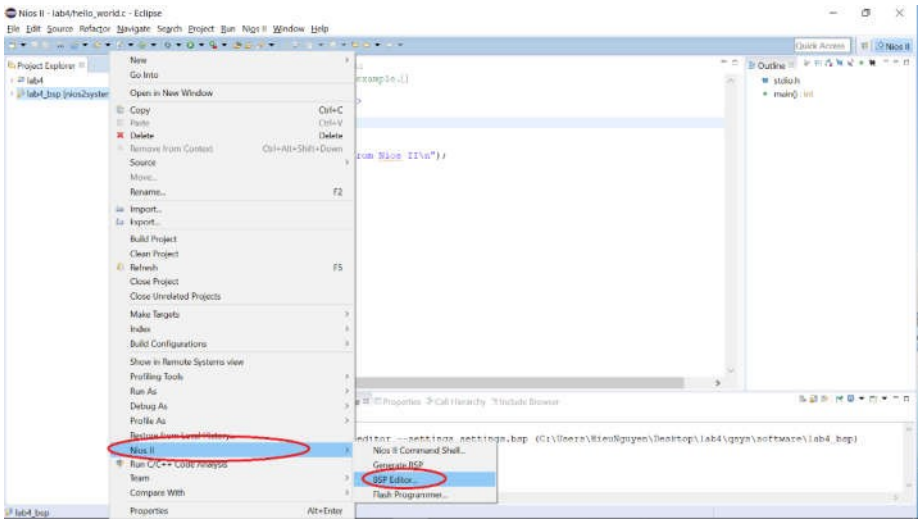
Doing all steps in LAB 4, we have a project with two folders: one is the project folder (the first folder) and two is the BSP (Board Support Package). The BSP is created automatically by Eclipse and Quartus Tool. This folder contains all nescessary libraries used for programming.

2. Knowledge of timer device

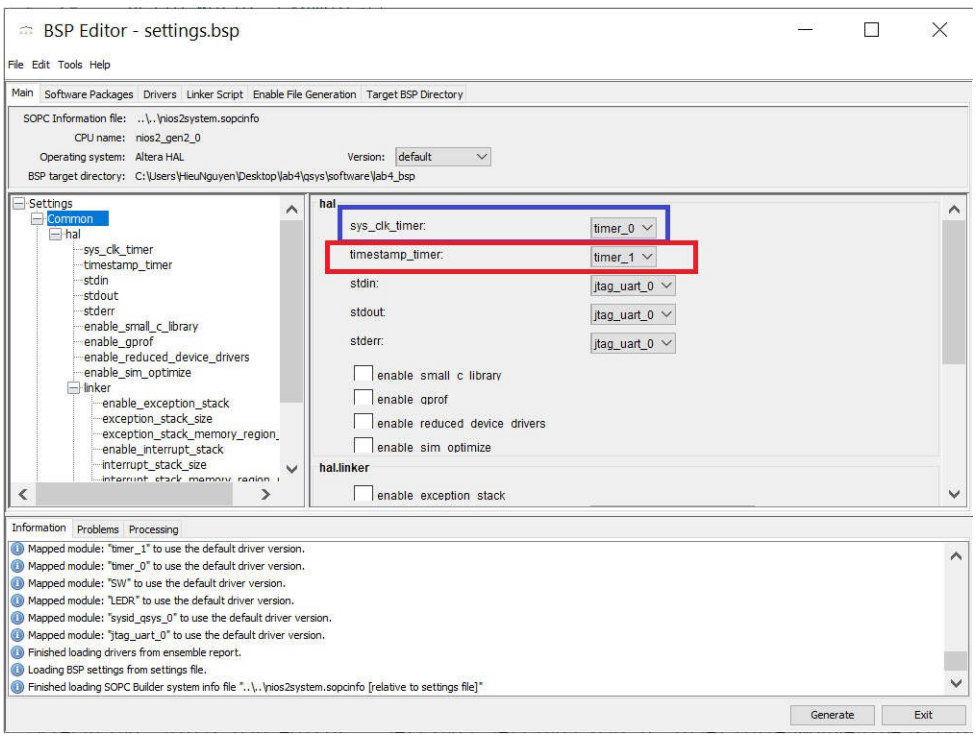
2.1 Design flow device

- Timer devices are hardware peripherals that count clock ticks and can generate periodic interrupt requests.
  - A timer device can be used to provide a number of time-related facilities, such as: HAL system clock, alarms, the time-of-day, and time measurement.
  - The HAL API (libraries of Nios II) provides two types of timer device drivers:
    - System clock driver: Supports alarms, such as you would use in a scheduler
    - Timestamp driver: Supports high-resolution time measurement
  - The HAL-specific API functions for accessing timer devices are defined in sys/alt_alarm.h and sys/alt_timestamp.h
  - The HAL system clock driver provides a periodic heartbeat, causing the system clock to increment on each beat.
  - Software can use the system clock facilities to execute functions at specified times, and to obtain timing information.
  - A specific hardware timer peripheral is defined as the system clock device within the BSP settings.
- Remember in lab4, we add two timer. In this project, we configure one of them become system clock timer and the other is used as timestamp timer. To do that, we need to define them in BSP settings following these steps.

- Right click on bsp, choose Nios II >> BSP Editor…



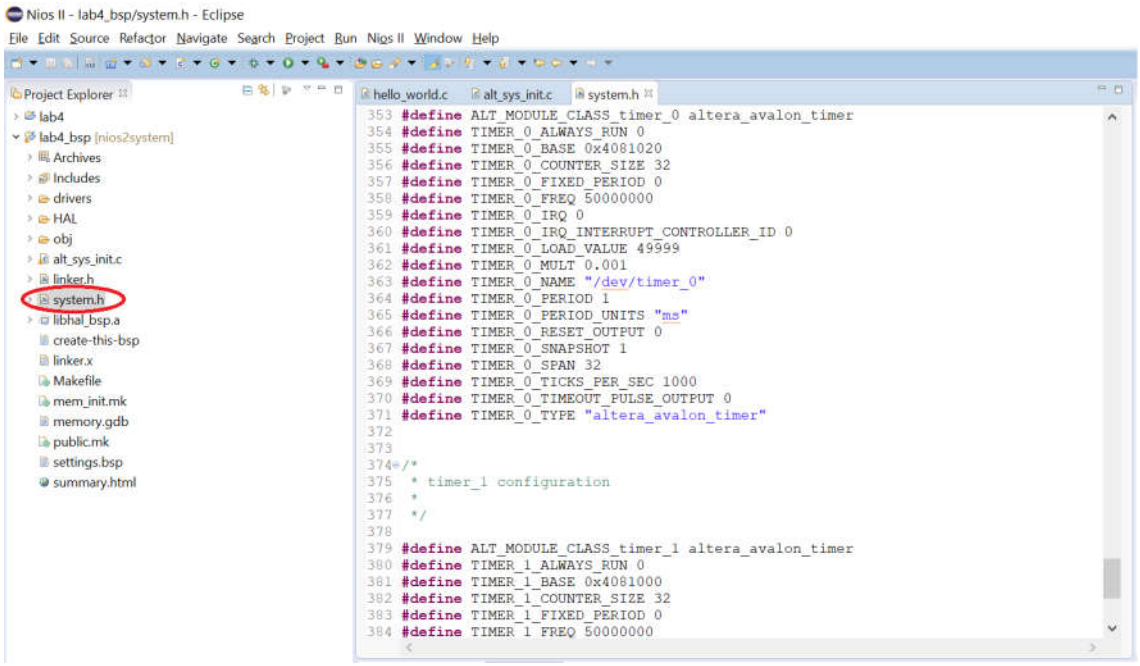
- In BSP Editor Window, choose one timer for sys_clk_timer and one timer for ***time_stamp_timer.***



- After that, click Generate.
- Right click on project and rebuild the project.

➢ After doing these step, for instance, in the picture timer_0 become sys_clk_timer and timer_1 become timestamp_timer.

➢ Next, in BSP, we considered file system.h. As disscussed in lab 2-1, this file contains all important parameters of the system: The frequency of CPU, name and address of each peripheral register, parameters relating to interrupt configuration, …. After adding timer to our system, the parameters of timers are also found here.


Nios II - lab4_bsp/system.h - Eclipse
File Edit Source Refactor Navigate Search Project Run Nios II Window Help
Project Explorer
lab4
lab4_bsp [nios2system]
Archives
Includes
drivers
HAL
obj
alt_sys_init.c
linker.h
system.h
libhal_bsp.a
create-this-bsp
linker.x
Makefile
mem_init.mk
memory.gdb
public.mk
settings.bsp
summary.html
hello_world.c
alt_sys_init.c
system.h
353 #define ALT_MODULE_CLASS_timer_0 altera_avalon_timer
354 #define TIMER_0_ALWAYS_RUN 0
355 #define TIMER_0_BASE 0x4081020
356 #define TIMER_0_COUNTER_SIZE 32
357 #define TIMER_0_FIXED_PERIOD 0
358 #define TIMER_0_FREQ 50000000
359 #define TIMER_0_IRQ 0
360 #define TIMER_0_IRQ_INTERRUPT_CONTROLLER_ID 0
361 #define TIMER_0_LOAD_VALUE 49999
362 #define TIMER_0_MULT 0.001
363 #define TIMER_0_NAME "/dev/timer_0"
364 #define TIMER_0_PERIOD 1
365 #define TIMER_0_PERIOD_UNITS "ms"
366 #define TIMER_0_RESET_OUTPUT 0
367 #define TIMER_0_SNAPSHOT 1
368 #define TIMER_0_SPAN 32
369 #define TIMER_0_TICKS_PER_SEC 1000
370 #define TIMER_0_TIMEOUT_PULSE_OUTPUT 0
371 #define TIMER_0_TYPE "altera_avalon_timer"
372
373
374 /*
375 * timer_1 configuration
376 *
377 */
378
379 #define ALT_MODULE_CLASS_timer_1 altera_avalon_timer
380 #define TIMER_1_ALWAYS_RUN 0
381 #define TIMER_1_BASE 0x4081000
382 #define TIMER_1_COUNTER_SIZE 32
383 #define TIMER_1_FIXED_PERIOD 0
384 #define TIMER_1_FREQ 50000000

3. Using Timestamp Driver

- When using the timer as a timestamp timer, you can measure the time with high accruracy. You may need to measure time intervals with a degree of accuracy greater than that provided by HAL system clock ticks.
    - The HAL provides high resolution timing functions using a timestamp driver.
    - A timestamp driver provides a monotonically increasing counter that you can sample to obtain timing information.
    - The HAL only supports one timestamp driver in the system.
    - The Altera-provided timestamp driver uses the timer that you specify.
    - If a timestamp driver is present, the following functions are available:
        - alt_timestamp_start()
        - alt_timestamp()
        - alt_timestamp_freq()
- The timestamp driver functions are defined (prototyped) in the alt_timestamp.h header file.
    - Calling alt_timestamp_start() starts the counter running
    - Subsequent calls to alt_timestamp() return the current value of the timestamp counter
    - Calling alt_timestamp_start() again resets the counter to zero
    - The behavior of the timestamp driver is undefined when the counter reaches the value (232 - 1)
    - Calling alt_timestamp_freq() return the rate at which the timestamp counter increments. This rate is typically the hardware frequency of the NIOS II processor system – Usually millions of cycles per second.

Example 1: This code is used to measure the time of a function processing and display this value to console.

```
#include <stdio.h>
#include <sys/alt_timestamp.h>

void func() {
      int a,b,c;
      c = a + b;
}

int main()
{
        if (alt_timestamp_start() < 0) {
              printf("Error !");
        }

        int time1;
        int time2;
        int time_base;

        time1 = alt_timestamp();
        time2 = alt_timestamp();
        time_base = time2 - time1;

        time1 = alt_timestamp();
        func();
        time2 = alt_timestamp();

        printf("Processing time of func() is %d \n",(float)(time2-time1-
time_base)/alt_timestamp_freq());

        return 0;
}
```

- In the program, time1 and time2 is used to save the current value of the timestamp counter. Because alt_timestamp() is also a function, it needs time to process which is saved by time_base.
- To measure the time process func(), we need to count the value of timestamp counter before and after func() excute. This results is time2-time1-time_base. It is the value measured by counter. To convert it to time, we need to devide this value by frequency of the timestamp counter.

Example 2: In lab 2-1, we know that there is a function call usleep() used to create delay time in microcomputer. In this example, we remesure the accruracy of this function by using the concept in example 1.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/alt_timestamp.h>

int main()
{
        if (alt_timestamp_start() < 0) {
              printf("Error !");
        }

        int time1;
        int time2;
        int time_base;

        time1 = alt_timestamp();
        time2 = alt_timestamp();
        time_base = time2 - time1;

        time1 = alt_timestamp();
        usleep(500000);
        time2 = alt_timestamp();

        printf("Processing time of usleep is %d \n",(float)(time2-time1-
time_base)/alt_timestamp_freq());

        return 0;
}
```

- The result shows that the actual processing time of ***usleep()*** function is longer approximately 40% the time we need.

4. Using Alarm Driver

- Alarms can be considered part of the exception processing system.
- User functions can be registered to be executed at a specified time using the HAL alarm facility.
- All the libraries of alarm driver is in **alt_alarm.h.**
- Software program registers an alarm by calling the function:

```
int alt_alarm_start (alt_alarm* the_alarm,
                     alt_u32    nticks,
                     alt_u32    (*callback) (void* context),
                     void*      context);
```

- The function callback() is called after nticks have elapsed.
- The input argument context is passed as the input argument to callback() when the call occurs:
  - The HAL does not use the context parameter
  - It is only used as a parameter to the callback() function
- User code must allocate the alt_alarm structure (init a variable in alt_alarm
- type), pointed to by the input argument alarm
  - This data structure must have a lifetime that is at least as long as that of the alarm.
  - The best way to allocate this structure is to declare it as a static or global.

- For a static variable, the last value of the variable is preserved between successive calls to a function.
  - The callback function can reset the alarm
  - The return value of the registered callback function is the number of ticks until the next call to callbac A return value of zero indicates that the alarm should be stopped
  - You can manually cancel an alarm by calling alt_alarm_stop()
  - One alarm is created for each call to alt_alarm_start()
  - Multiple alarms can run simultaneously

Example 3: This program is used to print alarm to console after a time.

```
#include <stdio.h>
#include <alt_types.h>
#include <sys/alt_alarm.h>

alt_u32 my_alarm_callback(void* context){
      // This function is called once per second
      printf("Alarm called\n");

      return alt_ticks_per_second();
}

int main()
{
      static alt_alarm alarm;

      if (alt_alarm_start(&alarm, alt_ticks_per_second(),
my_alarm_callback, NULL) < 0)
      {
      printf("No system clock available\n");
      }

      while (1){

      }
        return 0;
}
```

In main function we have while (1) loop which means the program will stuck here forever. The result shows that the sentence "Alarm called" is display every 1s. Therefore, the function my_alarm_callback(void* context) is called every 1s. To make it clear, the microcomputer execute while(1) loop if there is no signal from alarm. When the alarm counts enough ticks, it creates a signal to microcomputer. It makes the system changes to execute function my_alarm_callback(). Alarms can be considered part of the exception processing system.

III. Experiment

1. Build a Nios II system with timer. Write a C program to do this task by using alarm Each led from LED1 to 10 turns on in 500ms and then turn off.
2. Build a Nios II with timers. Write a C program to do these tasks:
   - When reset, all LEDs off.
   - When switch 1 is set, LED1 blink with frequency 0.25Hz, LED3 blink with frequency 0.5Hz, LED5 blink with frequency 1Hz, LED7 blink with frequency 2Hz, LED9 blink with frequency 4Hz.
   - When switch 1 is not set, all LEDs off.
3. Build a Nios II with timers. Write a C program to do these tasks by using alarm:
   - When switch 1 is set, each led at even position from LED1 to 10 turns on in 500ms and then turn off while each led at odd position from LED1 to 10 turns on in 1s and then turn off.
   - When switch 2 is set, each led from LED1 to 5 turns on in 500ms and then turn off. Each led from led 6 to 10 turn on in 1s and turn off.
   - When two switch are set, turn on all leds.
   - When two switch are not set, turn off all leds.